FLUX.2 Architectural & Urban Studio

A Technical White Paper for Reproducible AI-Assisted Design Workflows

Mustafa Raşit Şahin, Ph.D.

FLUX.2 Architectural & Urban Studio Team | February 2026

Repository: GitHub – FLUX.2 Architectural & Urban Studio

Abstract

Contemporary architectural and urban design practice increasingly depends on high-frequency visual iteration, yet many AI image-generation interfaces remain weak in reproducibility, process traceability, and operational transparency. The present paper introduces **FLUX.2 Architectural & Urban Studio** as a workflow-centric system that addresses this gap through explicit session-state contracts, model-aware control policies, queue-governed execution, and integrated safety and observability mechanisms. The methodological contribution is not a new generative model architecture per se, but a systems design in which design intent, runtime configuration, and operational evidence are coupled within a single interactive environment. Under that formulation, each generation request is treated as a structured experiment whose parameters can be reproduced, audited, and compared across scenarios. The discussion details architectural decomposition, workflow protocol, runtime model management, and reproducibility formalization, and it outlines evaluation criteria and validity boundaries for practical deployment in design teams.

Figure 1: Representative generation artifact (FLUX.2 Klein 4B, seed 1, 4 steps, guidance 1.0) positioned as a visual summary beneath the abstract.

1 Introduction

Architectural and urban design workflows are increasingly shaped by computational image synthesis, yet their decision quality still depends on process discipline rather than image novelty alone. Professional design teams do not merely ask whether an output is visually persuasive; they also ask whether the result is contextually grounded, reproducible under controlled settings, and auditable across iterative review cycles. These questions become critical in real projects where alternatives inform client

communication, internal design governance, and, in later phases, technical coordination with engineering constraints. In this setting, speed is necessary but insufficient: iterative throughput must be accompanied by explicit methodological traceability.

The dominant interaction model in many AI generation interfaces remains prompt-centric and single-shot in structure. Users iteratively rewrite prompts, sample outputs, and manually select candidates, but the surrounding state is often weakly formalized. Model identity, parameter defaults, seed strategy, and execution environment can change implicitly across reruns, making it difficult to establish whether output differences arise from design intent or from hidden runtime drift. As a consequence, seemingly successful ideation sessions can become analytically fragile when teams attempt to justify design choices or reproduce prior outcomes for comparison.

The analysis frames that gap as a design-computation systems problem. The central claim is that reliable AI-assisted architectural exploration requires a workflow architecture in which interaction state, model policy, and operational controls are explicitly coupled. FLUX.2 Architectural & Urban Studio is presented as an implementation of that thesis. The platform is not introduced primarily as a novel generative model, but as a controlled execution environment in which generation actions are embedded in stage-aware workflow logic, session-state contracts, and operational observability mechanisms.

To formalize the argument, a generation run can be represented as

$$y = F(p, m, \theta, s, e),$$

where p denotes prompt content, m denotes model identity, θ is the parameter vector, s is the seed policy realization, and e denotes the execution environment profile. The representation highlights a practical fact: reproducibility is a system-level property, not a prompt-level property. If any component of (p, m, θ, s, e) varies without explicit user intent, replay equivalence is no longer guaranteed and comparative reasoning between alternatives weakens.

FLUX.2 addresses this issue through a workflow-centric architecture with three methodological commitments. First, user interaction is constrained by explicit context-gating logic so that generation is anchored to project metadata rather than operating as a context-free sampling loop. Second, session-state transitions are governed through canonical defaults, model-aware presets, and recovery snapshots, reducing hidden drift across reruns. Third, runtime execution is operationalized through queue controls, health checks, and safety pipelines, ensuring that failure conditions are surfaced with actionable semantics rather than silent degradation.

Within this framing, the contributions of the paper are architectural, operational, and evaluative. Architecturally, the paper defines a layered decomposition that separates UI orchestration, state consistency, runtime services, and observability while preserving explicit interfaces. Operationally, it describes a local-first model resolution strategy and failure-containment mechanisms suitable for production-like iterative use. Evaluatively, it proposes a reproducibility-oriented measurement perspective in which latency, queue reliability, recovery behavior, and state integrity are treated as co-equal indicators.

The scope of the paper is intentionally bounded. FLUX.2 does not claim to solve all quality assessment challenges in computational design, nor does it present a complete multi-tenant SaaS governance plane. Instead, it focuses on a practical systems question: how to make AI-assisted design exploration methodologically dependable in single-team and limited-team contexts where traceability and iteration speed must coexist. Claims are therefore constrained to the behavior of the integrated workflow architecture rather than to universal model-quality superiority.

The remainder of the manuscript is organized as follows. Section 2 introduces conceptual background and clarifies why workflow-centric interaction is a more appropriate abstraction than single-shot prompting for traceable design reasoning. Section 3 presents the system architecture with layered responsibilities, interface dependencies, and failure-containment behavior. Subsequent sections describe workflow protocol, runtime and model management policy, reproducibility formalization, and safety/observability mechanisms. The paper then outlines an evaluation framework, discusses limitations and validity boundaries, and concludes with implications for future research and deployment practice.

2 Conceptual Background

Two paradigms dominate current AI image workflows in design contexts. The first is a single-shot interaction paradigm in which prompt refinement is iterative but weakly structured; this approach often maximizes short-term ideation speed while under-specifying state and provenance. The second is a workflow-centric paradigm in which context, control parameters, and execution records are coordinated explicitly across stages. FLUX.2 is positioned in the second paradigm, with the design assumption that reliable iteration requires formalized transitions between stages rather than unconstrained prompt cycling.

From a systems perspective, reproducibility in interactive generation differs from reproducibility in static batch experiments. Interactive environments introduce reruns, UI-triggered state mutations, fallback paths, and partial failures. Consequently, reproducibility must be defined not only at model inference time but across the entire request lifecycle, including state initialization, model resolution, queue dispatch, and artifact persistence. Such framing motivates the use of a session-state contract as a first-class abstraction rather than a purely implementation-level convenience.

3 System Architecture

The system architecture follows a layered model intended to isolate concerns while preserving explicit cross-layer contracts. Rather than coupling user controls directly to model execution routines, FLUX.2 interposes state and operational governance layers that regulate how requests are validated, transformed, and executed. Such design reduces hidden side effects and improves diagnosability when failures occur during iterative workflows.

Within that layered structure, the UI orchestration layer, implemented in `ui/`, governs workflow navigation, input capture, parameter interaction surfaces, and contextual feedback. Its interface obligation is to emit well-formed intent signals rather than invoke unconstrained runtime actions. Dependency management at that level relies on shared state contracts that determine admissible controls and enforce context gating, while failure containment uses explicit user-facing blockers and fallbacks so malformed requests do not propagate into runtime services.

The state consistency layer functions as the synchronization core. Canonical defaults, model-dependent parameter constraints, preset mappings, and replay-relevant metadata are maintained centrally to prevent silent mutation of critical execution variables during interactive reruns. Bidirectional dependencies connect that layer upward to widgets and downward to runtime invocations, and deterministic reapplication with recovery snapshots enables drift detection and correction without full session reset.

Runtime services, implemented primarily in `src/flux2/`, encapsulate model loading, generation, queue lifecycle management, safety checks, and artifact persistence. Responsibilities include local-first model resolution, parameterized inference, and bounded fallback under constrained resources. Contract-driven interfaces consume normalized payloads and return structured outputs or typed failure states, while stage-localized containment keeps model-availability failures, queue synchronization errors, and safety blocks analytically distinct instead of collapsing them into generic exceptions.

Operational and observability functions provide metrics, logging, health checks, and deployment-facing diagnostics. Semantic generation outcomes are not altered by that layer; instead, execution state becomes inspectable and recoverable through queue summaries, model health signals, and runtime telemetry events. Monitoring-informed recovery actions reduce mean time to diagnosis and lower repeated failure loops in long-running sessions. The following schema captures the layered topology and principal control/data pathways.

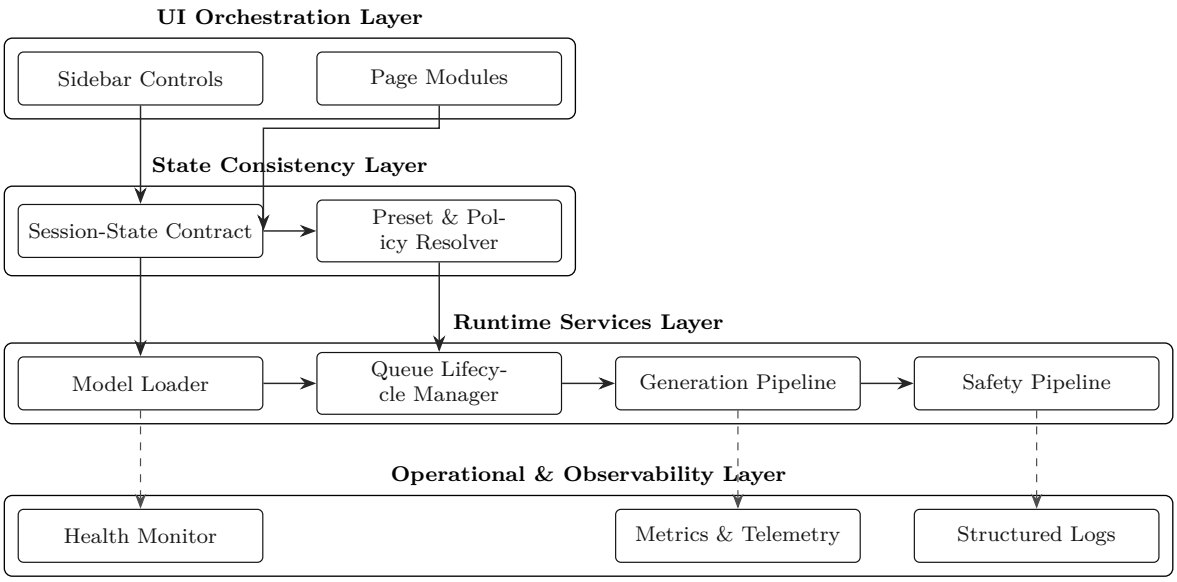

Figure 2: Layered FLUX.2 architecture showing control and observability paths.

The schema indicates a deliberate reduction of direct coupling between interface components and inference execution. UI modules do not directly control model internals; instead, intent is routed through the session-state contract and policy resolver, which acts as a normalization boundary before runtime invocation. That boundary is central to reliability because potentially inconsistent interactive input is transformed into constrained execution payloads. Runtime decomposition further isolates queue management, generation, and safety control so failures can be classified by stage rather than treated as monolithic errors. Observability components operate as orthogonal monitors that provide diagnosis and recovery evidence without altering generation semantics. Collectively, responsibility localization improves maintainability, and explicit failure paths improve operational robustness.

Architectural design targets four coupled objectives: traceability of design alternatives, stability of interactive state transitions, operational transparency during generation, and compatibility-preserving interface evolution. Coupling among objectives is explicit because throughput-only optimization can degrade reproducibility, while strict reproducibility constraints without observability can increase recovery overhead under failure. Layered decomposition therefore functions as a balancing mechanism across competing but interdependent requirements.

4 Workflow Model

The workflow is implemented as an explicit design protocol rather than a collection of independent UI features. Conceptually, each stage transforms a partial design state into a more constrained and better-documented state, and the output of one stage becomes the admissible input space for the next. The protocol begins with project setup and contextual framing, proceeds through massing and façade articulation, advances to scenario-level synthesis and view control, and concludes with comparative review and export. This stage ordering is not purely ergonomic; it is a control mechanism that limits semantic

drift between early intent and late-stage visual artifacts.

Information flow across stages follows a progressive constraint pattern. Early stages establish project identity, geography, and design-phase semantics; middle stages introduce geometric and material decisions; later stages focus on scenario comparison and communication outputs. Because these transitions are stateful, the workflow preserves provenance links between choices and outputs, making it possible to interpret variation as structured exploration instead of random sampling. In methodological terms, the protocol transforms generation from a prompt-only operation into a sequence of constrained decision updates.

Context gating remains the principal validity safeguard in the workflow. Generation controls are activated only when minimum project context is available, ensuring that runs are anchored to explicit assumptions rather than latent defaults. The mechanism reduces invalid requests and improves comparability between alternatives generated in different sessions. Formally, if $\mathcal{C}_t$ denotes context completeness at stage t and τ denotes the admissibility threshold, generation is enabled only when $\mathcal{C}_t \geq \tau$. Although the platform remains interactive, the gating policy inserts a lightweight methodological check before expensive runtime execution.

The protocol also contains explicit decision checkpoints at transitions where semantic commitment increases, including (i) project-context finalization, (ii) model/preset confirmation, (iii) scenario comparison selection, and (iv) export commitment. At each checkpoint, state snapshots are retained to support controlled rollback, enabling operational rollback after runtime or queue failure and epistemic rollback when earlier design branches require re-evaluation after downstream outcomes become available. Failure and revision are therefore modeled as expected workflow events rather than exceptional interruptions.

Guided workflow and legacy-compatible navigation operate under shared state contracts. Compatibility does not require identical interaction paths; instead, interoperability is preserved for critical state keys, parameter semantics, and runtime handoff payloads. Migration friction for existing users is reduced while protocol discipline remains intact for workflow-first usage. From a systems-maintenance perspective, compatibility mode acts as an interface-stability layer that decouples UI evolution from core runtime and reproducibility guarantees.

5 Model Runtime and Local-First Strategy

The runtime stack is designed to make model execution predictable under heterogeneous hardware and partial-resource conditions. A local-first resolution policy is used as the default operational mode: model readiness is determined from local artifact availability and explicit path contracts before any inference attempt is scheduled. The policy is significant because it converts model selection from a cosmetic UI choice into a verifiable runtime precondition, thereby reducing late-stage execution failures.

Runtime behavior can be described as a four-step control sequence: (1) resolve model identity and profile constraints, (2) validate local artifact availability and auxiliary dependencies, (3) apply model-specific defaults and fixed parameters, and (4) invoke queue-governed execution with bounded fallback handling. Because these steps are explicit, failure states are classifiable (missing model, invalid path, resource pressure, queue-state inconsistency) and can be mapped to targeted recovery actions.

The platform supports multiple FLUX.2 model classes with distinct latency–fidelity–resource trade-offs. Distilled families prioritize interactive throughput and low-step convergence, while base and larger families increase control range and visual fidelity at higher VRAM and runtime cost. Model-aware defaults are therefore policy decisions, not convenience presets: they reduce the probability that users apply unsuitable parameter regimes to incompatible model classes.

Let quality, latency, and resource terms be represented by (Q, L, R). A simple utility view for profile selection can be written as

$$U(m) = \alpha Q(m) - \beta L(m) - \gamma R(m),$$

where weights (α, β, γ) depend on project phase and hardware envelope. Early ideation typically increases β (latency sensitivity), whereas late-stage presentation preparation increases α (fidelity sensitivity).

Model Family	**Typical VRAM**	**Speed Tier**	**Control Regime**	**Primary Use**
FLUX.2 Klein 4B	∼8 GB	Fast	distilled / constrained	rapid iterative exploration
FLUX.2 Klein 9B	∼16 GB	Fast	distilled / higher capacity	detail-sensitive iteration
FLUX.2 Klein Base 4B	∼8 GB	Slower	broader guidance range	controllable studies on limited hardware
FLUX.2 Klein Base 9B	∼16 GB	Slower	broad control / high capacity	high-quality comparative design runs
FLUX.2 Dev	∼20 GB+	Slowest	maximum-capacity profile	final-fidelity or complex-scene generation

Table 1: Representative model profiles in FLUX.2 Architectural & Urban Studio.

The table should be interpreted as an operational policy map: model class determines not only expected visual behavior but also admissible parameter regimes and expected runtime stability envelope.

In local-only mode, binaries copied into expected `weights/` paths are auto-discovered and resolved without mandatory persistent overrides. If a selected model is absent, the interface does not silently hide it; instead, it marks the model as unavailable and provides explicit remediation instructions. The behavior preserves planning continuity by allowing users to inspect intended profile configurations while preventing invalid execution.

Fallback behavior is intentionally bounded. When runtime loading fails under specific conditions (for example, capacity mismatch in higher-parameter variants), the system may attempt policy-compatible fallback to a smaller class where appropriate; otherwise, execution is blocked with typed diagnostic feedback. The bounded strategy prevents uncontrolled degradation and keeps failure semantics interpretable for both users and operators.

6 Reproducibility and State Integrity: Formalization

The section formalizes reproducibility as a state-constrained systems property. The objective is to ensure that output variation reflects intentional design changes rather than latent runtime divergence. Three objects are treated as first-class controls: seed policy, parameter snapshot, and run state. Their joint behavior defines deterministic replay conditions and provides the basis for state-drift monitoring.

Let generation at run index k be represented by

$$y_k = F(p_k, m_k, \Theta_k, \sigma_k, e_k),$$

where p_k is prompt payload, m_k is model identifier, Θ_k is the parameter snapshot, σ_k is seed-policy output, and e_k is environment profile.

The seed policy is defined as a mapping

$$\sigma_k = \Pi_{\text{seed}}(\bar{s}, k, \eta),$$

where $\bar{s}$ is the base seed, k is run index, and η is policy mode metadata (for example fixed, offset, or schedule-based). The formulation separates user intent (base seed and policy mode) from concrete per-run seed realization.

The parameter snapshot is defined as

$$\Theta_k = (\theta_k, \phi_k, \kappa_k),$$

where θ_k denotes inference controls (e.g., steps, guidance), ϕ_k denotes workflow-context parameters, and κ_k denotes policy constraints (e.g., model-class caps and compatibility guards). A snapshot therefore captures both user-facing and system-imposed controls at execution time.

Run state is defined as

$$S_k = (p_k, m_k, \Theta_k, \sigma_k, e_k, q_k, a_k),$$

where q_k is queue lifecycle state and a_k is artifact metadata reference. The state tuple supports replay, audit, and failure analysis under a single contract.

Two runs i and j are replay-equivalent if

$$\mathcal{E}(i,j) = 1 \iff (p_i, m_i, \Theta_i, \sigma_i, e_i) = (p_j, m_j, \Theta_j, \sigma_j, e_j).$$

In practice, equality at this level defines deterministic intent-equivalence. Bounded numerical differences may still occur due to low-level hardware kernels; therefore, the condition is interpreted as a control-equivalence guarantee, not an unconditional bitwise-identity claim.

Let $\hat{S}_t$ be intended state at time t and $\tilde{S}_t$ be observed state. A normalized drift score is

$$D_t = \frac{\sum_{r\in\mathcal{V}} w_r \, d_r\left(\hat{S}_{t,r}, \tilde{S}_{t,r}\right)}{\sum_{r\in\mathcal{V}} w_r}, \qquad 0 \le D_t \le 1,$$

where $\mathcal{V}$ is the monitored variable set, $w_r \ge 0$ are importance weights, and d_r are variable-specific distance functions normalized to $[0,1]$. Operationally, D_t quantifies contract deviation intensity and enables threshold-based interventions.

A drift-triggered recovery rule can be written as

$$\text{Recover}_t = \mathbf{1}[D_t > \tau_D],$$

with threshold τ_D set by deployment policy. When triggered, recovery actions may include snapshot restoration, preset reapplication, queue-state reconciliation, or controlled fallback to compatible model profiles.

From an empirical standpoint, formalization also enables measurable audit workflows. Replay-equivalence rates, drift-threshold exceedance frequencies, and recovery-action distributions can be tracked longitudinally across releases. As a result, reproducibility governance moves from a qualitative aspiration to an operationally testable systems contract.

7 Safety, Governance, and Observability

Safety controls are integrated as staged checks in the generation lifecycle, covering both prompt-level and output-level risk filtering. The key architectural property is transparency: blocked actions produce explicit user-visible reasons rather than

silent rejection, which preserves procedural clarity and supports policy review.
Governance is supported through structured metadata persistence, report generation, and export pathways that retain execution context. Such support allows design alternatives to be interpreted as traceable decision artifacts rather than isolated images, improving auditability in collaborative review settings.
Operational monitoring is enabled through optional infrastructure assets for reverse-proxy deployment, queue and service orchestration, and metrics/alerting integration. The practical outcome is a tighter coupling between runtime execution and diagnosable system state, which shortens recovery cycles under partial failure conditions.

8 Deployment and Scalability Considerations

The platform supports both local service-style deployment and containerized operation. Scalability is primarily mediated by a queue-oriented execution model that allows worker expansion while preserving centralized control and monitoring surfaces. The architecture is suitable for incremental scaling strategies in which throughput is increased without discarding workflow governance constraints.
For production-oriented use, three controls are critical: explicit environment profiles for behavioral consistency, runtime/startup health checks for early failure detection, and strict secret/model-binary handling policies that prevent accidental leakage into version control.

9 Evaluation Methodology

The methodology is designed for internal or external technical review and is intentionally protocol-oriented. It defines how system behavior should be measured under controlled conditions without asserting outcome superiority in the absence of reported data. Evaluation is organized around five research questions: (RQ1) whether fixed state contracts improve replay consistency across repeated runs; (RQ2) how model profile selection affects latency and operational stability under fixed prompts; (RQ3) whether queue lifecycle controls reduce failure propagation during burst workloads; (RQ4) how recovery mechanisms affect service continuity after injected faults; and (RQ5) whether workflow context gating reduces invalid execution attempts relative to unconstrained interaction.
Independent factors include model class, hardware profile, queue load regime, safety policy level, and workflow mode (guided vs compatibility). Controlled variables include prompt suite version, seed policy, parameter snapshot schema, environment profile, and run ordering policy. To minimize confounding, each experimental block should preserve $(p, m, \Theta, \sigma, e)$ except for the factor under test, and all runs should log queue state transitions and recovery events. Prompt suites should be stratified by design intent rather than random collection; a recommended structure includes baseline architectural massing prompts, façade/material articulation prompts, scenario-comparison prompts, and edge-case prompts that stress queue or memory behavior. Each stratum should include lexical variants and context-completeness variants so robustness can be assessed under semantically similar yet syntactically diverse inputs. Prompt-suite versioning must remain immutable during a reporting cycle.
Latency should be reported as distributional statistics (median, interquartile range, and high-percentile tails). Consistency should be assessed through replay-equivalence rate under fixed state tuples. Reliability should include queue completion ratio and failure propagation rate. Recovery should include mean recovery time and successful restoration ratio after injected fault scenarios. A compact metric set is

$$C = \frac{N_{\text{completed}}}{N_{\text{submitted}}}, \qquad \text{MRT} = \frac{1}{K}\sum_{k=1}^{K}\left(t_k^{\text{recovered}} - t_k^{\text{failed}}\right), \qquad \text{FPR} = \frac{\sum_{u=1}^{U} I_u^{\text{prop}}}{\sum_{u=1}^{U} I_u^{\text{root}}},$$

where I_u^{root} indicates a root incident and I_u^{prop} indicates incidents causally propagated from earlier failures. Operationally, C quantifies end-to-end completion robustness, MRT quantifies restoration responsiveness, and FPR quantifies containment quality in queue lifecycle management. Additional reporting should include replay-consistency score and drift-threshold exceedance frequency.
All reported metrics should include sample counts, confidence intervals or bootstrap uncertainty bounds, and stratified summaries by model class and hardware profile. Outlier policy must be declared a priori (retain, winsorize, or annotate) and applied consistently. Comparative claims should be limited to conditions with matched controls and sufficient sample size. Where assumptions for parametric tests are not satisfied, non-parametric alternatives should be preferred.
Each experiment must persist reproducibility metadata including prompt payload hash, model identifier, parameter snapshot, seed-policy configuration, environment profile, queue trace, and artifact reference. Reproduction should be attempted in at least two independent sessions using identical metadata manifests. Any divergence must be classified as either expected bounded numerical variance or state-contract violation, with corrective action records attached.
Methodological rigor further requires transparent reporting of exclusion criteria, rerun policies, and environment drift controls to preserve interpretability across review cycles. Such reporting ensures that statistical summaries remain attributable to protocol choices rather than undocumented execution variance.

10 Threats to Validity

Internal validity may be threatened by uncontrolled background processes, inconsistent warm-start states, or hidden parameter mutations across reruns. For example, asynchronous cache state changes can alter observed latency independently

of model choice. Mitigation includes pre-run environment stabilization, explicit warmup protocols, immutable parameter snapshots, and per-run validation of key state variables before dispatch.

External validity is constrained when results obtained on a limited set of GPUs, operating systems, or project types are generalized to broader deployment contexts. A concrete risk is overgeneralizing recovery behavior measured on a single workstation to multi-worker production environments. Mitigation requires cross-hardware replication, workload diversification across project scales, and explicit reporting of deployment envelope boundaries.

Construct validity can be compromised when selected metrics do not faithfully represent the target quality construct. For instance, low latency alone does not indicate design usefulness or methodological reliability. Mitigation involves multi-metric triangulation (latency, replay consistency, queue reliability, recovery quality), alignment checks between metric definitions and research questions, and periodic review of metric relevance with domain stakeholders.

Operational validity is threatened when benchmark procedures diverge from real usage conditions, such as synthetic prompts that do not reflect iterative design workflows or fault-injection rates that are unrealistically high. Mitigation includes scenario-grounded prompt strata, mixed workload schedules that mirror session behavior, and calibration of failure injections against observed operational incident frequencies.

11 Schema Artifacts (TikZ)

To improve publication quality and visual continuity, the section provides refined TikZ schematics for execution lifecycle and recovery logic. The primary layered architecture is already presented in Figure 2; it is not repeated here to avoid redundancy and visual crowding.

The first schematic details lifecycle progression from request submission to export delivery.

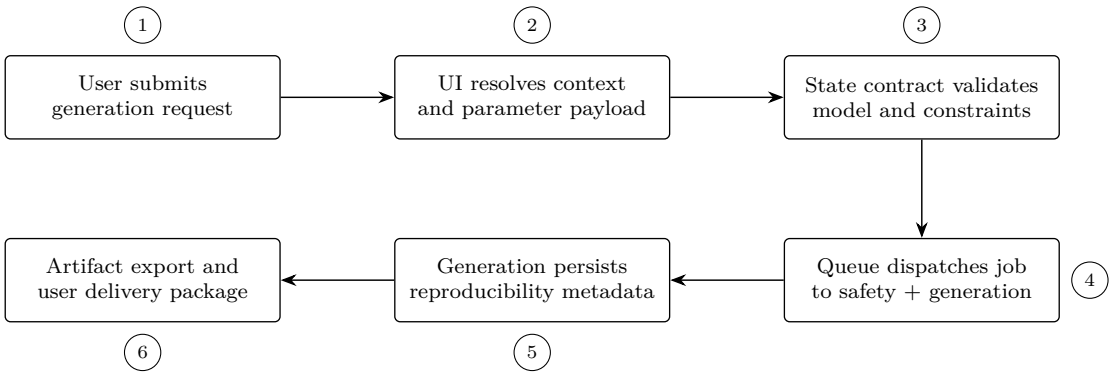

Figure 3: Execution lifecycle schematic with numbered stages and non-overlapping control flow.

Reproducibility guarantees are enforced at three control points: parameter resolution in the session-state contract, model-availability validation before queue dispatch, and metadata persistence at generation completion. The first point constrains execution inputs to an explicit snapshot, the second prevents hidden runtime substitution, and the third binds artifacts to replay-relevant metadata for audit and reproduction.

The second schematic formalizes bounded recovery behavior under failure conditions.

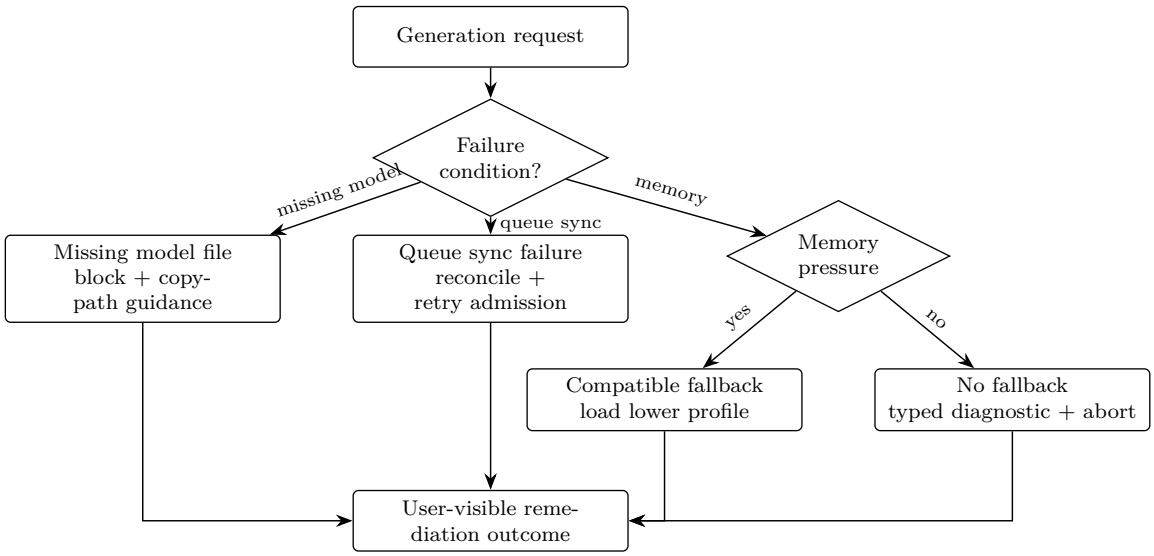

Figure 4: Failure-recovery schematic with bounded fallback and explicit user-facing outcomes.

Operationally, the recovery diagram specifies a bounded-fallback policy: failures are classified, mapped to explicit remediation logic, and surfaced with user-visible outcomes. The design improves incident containment by avoiding implicit degradation

paths and provides operations teams with predictable recovery semantics for runbook design.

12 Style Enforcement Notes

Stylistic and structural normalization has been performed with a claim–mechanism–evidence discipline that aligns narrative flow with technical accountability. Paragraph continuity was prioritized over list-fragmented exposition, terminology was harmonized across sections (session-state contract, queue lifecycle, local-first model resolution), and absolute wording was replaced with bounded language where empirical support is not yet complete. Mathematical formalization now anchors core claims through explicit mappings for deterministic replay, state drift, and queue reliability, thereby reducing interpretive ambiguity in methodological sections.

The revised manuscript also introduces stronger epistemic separation between implemented behavior, expected behavior, and experimentally verified behavior. Implemented behavior is described through architecture and control contracts; expected behavior is framed through policy-consistent runtime and recovery logic; experimentally verified behavior is restricted to protocol definitions and metric plans. Such separation improves scientific readability because reviewers can distinguish engineering capability from validated evidence without inferring unsupported performance claims.

Three principal evidence gaps remain before a fully empirical manuscript can be claimed. First, quantitative benchmark datasets are still required for latency strata, replay-consistency distributions, and queue recovery outcomes across heterogeneous hardware envelopes. Second, mitigation efficacy for identified validity threats has not yet been statistically characterized under controlled fault-injection regimes with reported confidence bounds. Third, external-validity support requires replication beyond the current deployment envelope, including multi-team, multi-environment studies with standardized prompt suites and comparable observability traces. These limitations do not negate the systems contribution, but they bound inferential strength to architectural coherence and methodological rigor rather than comparative performance superiority.

13 Future Work

Future work is organized around three research tracks: governance scalability, evaluation automation, and decision-coupled design analytics. Governance scalability requires explicit multi-user semantics for role-aware state control, conflict resolution under concurrent edits, and policy-auditable handoff protocols across collaborative review sessions. Evaluation automation requires standardized experiment orchestration that can execute prompt-stratified benchmark suites, collect reproducibility metadata, and generate confidence-aware summaries without manual post-processing bottlenecks.

Decision-coupled analytics should connect generated alternatives to scenario-level reasoning artifacts, including traceable rationale tags, comparative uncertainty annotations, and project-phase aware selection criteria. A high-impact extension is the introduction of reproducibility manifests as release-grade artifacts that unify configuration lineage, runtime traces, and recovery events within a machine-verifiable package. With that mechanism, operational claims and experimental claims can be inspected under a shared evidence contract.

Methodologically, subsequent studies should test whether stronger governance semantics and formalized evidence packaging improve reviewer agreement, reduce rerun divergence, and shorten diagnosis time under controlled failure injections. Establishing these causal relationships would convert currently plausible systems hypotheses into statistically supported design principles for reproducible AI-assisted design platforms.

14 Conclusion

FLUX.2 Architectural & Urban Studio demonstrates that AI-assisted design interfaces can be engineered as reproducibility-oriented workflow systems rather than prompt-only generation surfaces. The central contribution is architectural: session-state contracts, model-aware runtime policy, queue-governed execution, and observability instrumentation are integrated into a single control plane that preserves traceability across iterative design exploration. In that formulation, generation is treated as a governed experimental act rather than a loosely coupled sampling event.

The manuscript advances a methodological position in which reproducibility, state integrity, and recovery behavior are first-class evaluation dimensions alongside latency and visual fidelity. Formal definitions for replay equivalence and drift monitoring provide analyzable control targets, while protocol-oriented evaluation and validity framing define how claims should be tested under constrained assumptions. As a result, the work contributes both an implementation-level system and a review-ready methodological template for technical assessment.

Claim boundaries remain explicit: broad comparative superiority is not asserted in the absence of completed benchmark campaigns and cross-environment replications. Even under those boundaries, the presented architecture establishes a credible foundation for reliable, auditable, and operationally diagnosable AI-assisted architectural and urban design workflows. Continued empirical validation is expected to refine quantitative conclusions, but the systems thesis—that methodological reliability must be co-designed with generative capability—remains strongly supported by the integrated structure presented in this paper.

Repository: `https://github.com/mustafaras/flux2_architectural_urban_studio`